

Conception et Déploiement d'une API de Gestion d'Employés avec FastAPI, PostgreSQL et Docker

Documentation Pédagogique

16 juin 2026

Table des matières

1	Vue d'ensemble du projet	3
1.1	Opérations disponibles (CRUD)	3
2	Les technologies utilisées	3
2.1	FastAPI - Le framework web	3
2.2	SQLAlchemy - L'ORM	3
2.3	PostgreSQL - La base de données	3
2.4	Docker - La conteneurisation	4
3	Architecture détaillée	4
3.1	Architecture en couches	4
3.1.1	Couche 1 : La base de données (PostgreSQL)	4
3.1.2	Couche 2 : Les modèles (SQLAlchemy)	4
3.1.3	Couche 3 : Les schémas (Pydantic)	4
3.1.4	Couche 4 : Le CRUD	4
3.1.5	Couche 5 : Les routes (FastAPI)	4
3.1.6	Couche 6 : Le client (Swagger/Postman)	4
4	Installation de FastAPI	5
4.1	Option 1 : Installation globale (non recommandée)	5
4.2	Option 2 : Environnement virtuel (Recommandé)	5
4.3	Dépendances principales	5

5	Explication fichier par fichier	6
5.1	app/database.py - Connexion à la base de données	6
5.2	app/models.py - Structure de la table	7
5.3	app/schemas.py - Validation des données	7
5.4	app/crud.py - Opérations sur la base de données	8
5.4.1	Lecture de tous les employés	8
5.4.2	Lecture d'un employé par ID	8
5.4.3	Recherche par matricule	9
5.4.4	Création d'un employé	9
5.4.5	Mise à jour d'un employé	9
5.4.6	Suppression d'un employé	10
5.5	app/main.py - L'application FastAPI	10
5.5.1	Initialisation	10
5.5.2	Dépendance pour la base de données	10
5.5.3	POST /employees/ - Création	11
5.5.4	GET /employees/ - Liste	11
5.5.5	GET /employees/{employee_id} - Détail	11
5.5.6	DELETE /employees/{employee_id} - Suppression	12
6	La partie Docker	12
6.1	Dockerfile	12
6.2	docker-compose.yml	13
6.2.1	Service PostgreSQL	13
6.2.2	Service Backend	13
7	Flux de données complet	13
8	Commandes utiles	14
8.1	Lancement du projet	14
8.2	Accès à la documentation	14
9	Conclusion	14

Vue d'ensemble du projet

Ce projet est une API REST (Interface de Programmation d'Application) qui suit l'architecture REST et permet de gérer des employés.

Opérations disponibles (CRUD)

- ✓ **Créer** un employé
- ✓ **Lire** la liste des employés
- ✓ **Lire** un employé spécifique
- ✓ **Modifier** un employé
- ✓ **Supprimer** un employé

Les technologies utilisées

FastAPI - Le framework web

C'est le cœur de l'application. FastAPI est un framework Python moderne pour créer des APIs.

Avantages :

- + **Très rapide** : Performances comparables à Node.js ou Go
- + **Automatique** : Génère automatiquement une documentation interactive (Swagger) à l'adresse /docs
- + **Validation intégrée** : Validation des données avec Pydantic

SQLAlchemy - L'ORM

ORM = Object-Relational Mapping. Outil qui permet de manipuler la base de données en utilisant du Python.

Fonctionnalités :

- + Manipuler la base de données sans écrire de SQL brut
- + Créer des tables à partir de classes Python
- + Faire des requêtes avec une syntaxe Python intuitive

PostgreSQL - La base de données

Base de données relationnelle puissante et open-source qui stocke les données de façon permanente.

Docker - La conteneurisation

Permet d'empaqueter l'application et ses dépendances dans des "conteneurs" isolés.

Avantages :

- + **Reproductibilité** : Fonctionne pareil sur tous les ordinateurs
- + **Isolation** : Indépendant de la machine hôte
- + **Simplicité** : Déploiement facilité

Architecture détaillée

Architecture en couches

Couche 6 : Client (Swagger/Postman) ↑↓
Couche 5 : Routes (FastAPI) ↑↓
Couche 4 : CRUD (Opérations) ↑↓
Couche 3 : Schémas (Pydantic) ↑↓
Couche 2 : Modèles (SQLAlchemy) ↑↓
Couche 1 : Base de données (PostgreSQL)

Couche 1 : La base de données (PostgreSQL)

Stocke physiquement les données dans des tables.

Couche 2 : Les modèles (SQLAlchemy)

Définissent la structure des tables en Python.

Couche 3 : Les schémas (Pydantic)

Définissent la forme des données qui entrent et sortent de l'API.

Couche 4 : Le CRUD

Contient la logique d'interaction avec la base de données.

Couche 5 : Les routes (FastAPI)

Points d'entrée de l'API que le client appelle.

Couche 6 : Le client (Swagger/Postman)

Interface qui permet d'interagir avec l'API.

Installation de FastAPI

Option 1 : Installation globale (non recommandée)

```
pip install fastapi uvicorn sqlalchemy psycpg2-binary pydantic pytest
```

Cette méthode installe les dépendances directement sur le système. Elle peut provoquer des conflits de versions.

Option 2 : Environnement virtuel (Recommandé)

```
mkdir backend-gestion-employe
cd backend-gestion-employe

python -m venv venv

# Windows
venv\Scripts\activate

# Linux / Mac
source venv/bin/activate

pip install fastapi uvicorn sqlalchemy psycpg2-binary pydantic pytest

pip freeze > requirements.txt
```

Cette approche permet d'isoler les dépendances et correspond aux bonnes pratiques DevOps.

Dépendances principales

Chaque dépendance joue un rôle précis dans l'architecture de l'API.

- **FastAPI** Framework web permettant de créer les routes API, gérer les requêtes HTTP et générer automatiquement la documentation Swagger.
- **Uvicorn** Serveur ASGI qui exécute l'application FastAPI et écoute les requêtes HTTP.
- **SQLAlchemy** ORM permettant de manipuler la base de données PostgreSQL avec des classes Python.
- **psycpg2-binary** Driver PostgreSQL utilisé par SQLAlchemy pour établir la connexion à la base.
- **Pydantic** Bibliothèque de validation des données entrantes et sortantes de l'API.
- **Pytest** Framework de tests utilisé pour automatiser la validation du code (CI/CD).

Chaîne d'exécution :

Client → Uvicorn → FastAPI → Pydantic → SQLAlchemy → psycopg2 → PostgreSQL

Explication fichier par fichier

app/database.py - Connexion à la base de données

```
import os
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

# L'URL de connexion à la base de données
SQLALCHEMY_DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "postgresql://admin:admin123@localhost:5432/gptdb"
)
```

- i** `os.getenv()` : Récupère une variable d'environnement. Évite d'écrire les mots de passe en dur.
- i** **Structure de l'URL** : `postgresql://utilisateur:motdepasse@hôte:port/nom_de_la_base`
- i** **Valeur par défaut** : Pratique pour le développement local

```
engine = create_engine(SQLALCHEMY_DATABASE_URL)
```

engine : Point de connexion central à la base de données. Gère un pool de connexions.

```
SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)
```

SessionLocal : Fabrique de sessions (une session = conversation avec la base de données).

- i** `autocommit=False` : Validation manuelle des transactions (`commit()`)
- i** `autoflush=False` : Contrôle manuel de l'envoi des données

```
Base = declarative_base()
```

Base : Classe de base pour tous les modèles. Toutes les classes qui représenteront des tables hériteront de celle-ci.

app/models.py - Structure de la table

```
from sqlalchemy import Column, Integer, String
from .database import Base

class Employee(Base):
    __tablename__ = "employees"

    id = Column(Integer, primary_key=True, index=True)
    matricule = Column(String, unique=True, index=True, nullable=False)
    nom = Column(String, index=True)
    poste = Column(String)
    salaire = Column(Integer)
```

- ⚙ **class Employee(Base)** : Représente la table "employees" en Python
- ⚙ **__tablename__** : Nom de la table dans la base

Colonnes :

id Clé primaire, auto-générée, indexée

matricule — `unique=True` : Empêche les doublons

— `index=True` : Optimise les recherches

— `nullable=False` : Champ obligatoire

nom Indexé pour les recherches par nom

poste Simple champ texte

salaire Nombre entier (évite les problèmes de flottants)

app/schemas.py - Validation des données

```
from pydantic import BaseModel, Field

class EmployeeBase(BaseModel):
    matricule: str = Field(..., example="EMP-2026-001")
    nom: str
    poste: str
    salaire: int
```

EmployeeBase : Schéma de base avec tous les champs communs.

- 📘 **Field(...)** :
 - `...` : Champ obligatoire

— exemple : Exemple pour la documentation Swagger

```
class EmployeeCreate(EmployeeBase):  
    pass
```

Hérite de EmployeeBase, utilisé pour la création.

```
class EmployeeResponse(EmployeeBase):  
    id: int  
  
    class Config:  
        from_attributes = True
```

- **EmployeeResponse** : Renvoyé au client (avec l'ID)
- **from_attributes = True** : Permet de lire les données depuis SQLAlchemy

app/crud.py - Opérations sur la base de données

Lecture de tous les employés

```
def get_employees(db: Session):  
    return db.query(models.Employee).all()
```

- **db.query(...)** : Crée une requête SELECT
- **.all()** : Exécute et retourne tous les résultats

Lecture d'un employé par ID

```
def get_employee(db: Session, employee_id: int):  
    return db.query(models.Employee).filter(  
        models.Employee.id == employee_id  
    ).first()
```

- **.filter()** : Ajoute une condition WHERE
- **.first()** : Premier résultat (ou None)

Recherche par matricule

```
def get_employee_by_matricule(db: Session, matricule: str):  
    return db.query(models.Employee).filter(  
        models.Employee.matricule == matricule  
    ).first()
```

Utilisé pour vérifier les doublons.

Création d'un employé

```
def create_employee(db: Session, employee: schemas.EmployeeCreate):  
    db_employee = models.Employee(**employee.dict())  
    db.add(db_employee)  
    db.commit()  
    db.refresh(db_employee)  
    return db_employee
```

1. `employee.dict()` : Convertit l'objet Pydantic en dictionnaire
2. `**...` : Décompresse le dictionnaire en arguments
3. `db.add()` : Ajoute à la session (INSERT en attente)
4. `db.commit()` : Valide la transaction (exécute l'INSERT)
5. `db.refresh()` : Recharge pour récupérer l'ID généré

Mise à jour d'un employé

```
def update_employee(db: Session, employee_id: int, employee:  
    → schemas.EmployeeCreate):  
    db_employee = get_employee(db, employee_id)  
    if db_employee:  
        for key, value in employee.dict().items():  
            setattr(db_employee, key, value)  
        db.commit()  
        db.refresh(db_employee)  
    return db_employee
```

1. Récupère l'employé existant
2. Met à jour chaque champ
3. Valide la transaction

Suppression d'un employé

```
def delete_employee(db: Session, employee_id: int):  
    db_employee = get_employee(db, employee_id)  
    if db_employee:  
        db.delete(db_employee)  
        db.commit()  
    return db_employee
```

app/main.py - L'application FastAPI

Initialisation

```
Base.metadata.create_all(bind=engine)  
app = FastAPI(title="Gestion des Employés")
```

- Crée les tables si elles n'existent pas
- Initialise l'application FastAPI

Dépendance pour la base de données

```
def get_db():  
    db = SessionLocal()  
    try:  
        yield db  
    finally:  
        db.close()
```

- Crée une session par requête
- `yield` : Fournit la session à la route
- `finally` : Ferme la session même en cas d'erreur

POST /employees/ - Création

```
@app.post("/employees/", response_model=schemas.EmployeeResponse)
def create_employee(employee: schemas.EmployeeCreate,
                    db: Session = Depends(get_db)):
    existing = crud.get_employee_by_matricule(db, employee.matricule)
    if existing:
        raise HTTPException(
            status_code=400,
            detail="Matricule déjà utilisé."
        )
    return crud.create_employee(db, employee)
```

- `response_model` : Formate la réponse
- `employee: schemas.EmployeeCreate` : Validation automatique
- `db: Session = Depends(get_db)` : Injection de dépendance
- **Vérification** : Empêche les doublons de matricule
- **HTTP 400** : Mauvaise requête si matricule existe

GET /employees/ - Liste

```
@app.get("/employees/", response_model=list[schemas.EmployeeResponse])
def read_employees(db: Session = Depends(get_db)):
    return crud.get_employees(db)
```

GET /employees/{employee_id} - Détail

```
@app.get("/employees/{employee_id}",
         response_model=schemas.EmployeeResponse)
def read_employee(employee_id: int,
                  db: Session = Depends(get_db)):
    db_employee = crud.get_employee(db, employee_id)
    if db_employee is None:
        raise HTTPException(status_code=404,
                            detail="Employé introuvable")
    return db_employee
```

- `{employee_id}` : Paramètre d'URL
- `employee_id: int` : Validation du type
- **HTTP 404** : Non trouvé

DELETE /employees/{employee_id} - Suppression

```
@app.delete("/employees/{employee_id}")
def delete_employee(employee_id: int,
                    db: Session = Depends(get_db)):
    crud.delete_employee(db, employee_id)
    return {"message": "Employé supprimé"}
```

La partie Docker

Dockerfile

```
FROM python:3.11-slim
```

Image de base : Python 3.11 en version légère.

```
WORKDIR /app
```

Définit le répertoire de travail.

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

- Copie le fichier des dépendances
- Installe les dépendances (sans cache)

```
COPY ./app ./app
```

Copie le code de l'application.

```
CMD ["uvicorn", "app.main:app",
    "--host", "0.0.0.0",
    "--port", "8000"]
```

- Commande au démarrage
- `app.main:app` : Application FastAPI

- 0.0.0.0 : Écoute sur toutes les interfaces
- 8000 : Port d'écoute

docker-compose.yml

Service PostgreSQL

```
db-service:
  image: postgres:16
  environment:
    POSTGRES_DB: gptdb
    POSTGRES_USER: admin
    POSTGRES_PASSWORD: admin123
  ports:
    - "5432:5432"
  volumes:
    - gptdata:/var/lib/postgresql/data
```

- Image officielle PostgreSQL 16
- Configuration via variables d'environnement
- Port exposé
- Volume pour persister les données

Service Backend

```
backend-service:
  build: .
  ports:
    - "8000:8000"
  environment:
    DATABASE_URL: postgresql://admin:admin123@db-service:5432/gptdb
  depends_on:
    - db-service
```

- Build à partir du Dockerfile
- Port exposé
- URL de connexion (notez `db-service` comme hôte)
- Dépendance PostgreSQL

Flux de données complet

1. **Client** : Envoie une requête POST à `http://localhost:8000/employees/`

2. **FastAPI** : Reçoit la requête, l'aiguille vers `create_employee`
3. **Validation** : Vérifie que le corps correspond à `EmployeeCreate`
4. **Dépendance** : Appelle `get_db()` pour une session DB
5. **Logique métier** :
 - Vérifie si le matricule existe avec `get_employee_by_matricule`
 - Si non, appelle `create_employee`
6. **CRUD** :
 - Crée un objet SQLAlchemy
 - Ajoute à la session
 - Valide (`commit`)
 - Rafraîchit pour l'ID
7. **Réponse** : Convertit en `EmployeeResponse` (via `from_attributes=True`)
8. **Client** : Reçoit l'employé créé avec son ID

Commandes utiles

Lancement du projet

```
# Démarrer avec Docker Compose
docker compose up --build

# Arrêter
docker compose down

# Voir les logs
docker compose logs -f
```

Accès à la documentation

<http://localhost:8000/docs>

Conclusion

- ✓ **Isolation** : Environnement virtuel Python
- ✓ **Conteneurisation** : Docker pour la reproductibilité
- ✓ **Configuration** : Variables d'environnement
- ✓ **Architecture** : Séparation des préoccupations
- ✓ **Documentation** : Swagger automatique
- ✓ **Validation** : Pydantic à l'entrée/sortie

- ✓ Gestion d'erreurs : Codes HTTP appropriés

Fin de la documentation